Assignment 1

Prompt

For both models, I copied the requirements directly from the instructions

"Objective Write a C program that picks a secret number between 1 and 10. The user gets up to 3 tries to guess the number. Requirements • Use a fixed secret number (for example, 7). • Use printf to prompt the user and scanf to read their guess. • Use a while or for loop to let the user guess up to 3 times. • Use if-else statements to check if the guess is too high, too low, or correct. • Stop the loop immediately if the user guesses the correct number. • Print a final win or lose message. Example Output text Guess a number between 1 and 10. Attempt 1/3. Enter your guess: 5 Too low! Try again. Attempt 2/3. Enter your guess: 8 Too high! Try again. Attempt 3/3. Enter your guess: 7 Correct! You win!"

Gemini Code

```
/*
================================================================================
Name        : EECS 348 Assignment 1
Author      : [Your Full Name]
Description : C program that picks a secret number between 1 and 10 and
              allows the user up to 3 tries to guess it correctly.
Inputs      : User guess (integer entered via standard input).
Output      : Terminal output prompting the user and providing feedback.
Collaborators: None
Other Sources: None (Generated code improved for submission)
Creation Date: September 8, 2026
Revision Date: September 8, 2026
Revisions   : Initial implementation with complete comments.
================================================================================
*/

#include <stdio.h>

int main() {
    // Author: [Your Full Name]
    // Declare the fixed secret number to be guessed (7)
    const int SECRET_NUMBER = 7;

    // Declare variable to hold the user's input guess
    int guess = 0;

    // Flag to track whether the user successfully guessed the number (0 = false,
1 = true)
    int won = 0;

    // Print initial welcoming message to instruct the user
    printf("Guess a number between 1 and 10.\n");
```

```
    // Loop through up to 3 attempts (attempt numbers 1, 2, and 3)
    for (int attempt = 1; attempt <= 3; attempt++) {
        // Author: [Your Full Name]
        // Prompt user for their current guess with attempt count displayed
        printf("Attempt %d/3. Enter your guess: ", attempt);

        // Read the integer input entered by the user
        scanf("%d", &guess);

        // Check if the user's guess matches the secret number
        if (guess == SECRET_NUMBER) {
            // Print success message
            printf("Correct! You win!\n");

            // Set winning flag to true
            won = 1;

            // Break out of the loop immediately as required upon a correct guess
            break;
        }
        // Check if the guess is less than the secret number
        else if (guess < SECRET_NUMBER) {
            // Inform the user their guess was too low
            printf("Too low! Try again.\n");
        }
        // Otherwise, the guess must be greater than the secret number
        else {
            // Inform the user their guess was too high
            printf("Too high! Try again.\n");
        }
    }

    // Author: [Your Full Name]
    // If after 3 attempts the user did not guess correctly, display losing
message
    if (!won) {
        printf("Sorry, you lost! The secret number was %d.\n", SECRET_NUMBER);
    }

    // Return 0 to signal successful program execution
    return 0;
}
```

ChatGPT Code

```
#include <stdio.h>

int main() {
    int secretNumber = 7;
    int guess;
    int attempts = 0;
```

```
    printf("Guess a number between 1 and 10.\n");

    while (attempts < 3) {
        attempts++;

        printf("Attempt %d/3. Enter your guess: ", attempts);
        scanf("%d", &guess);

        if (guess == secretNumber) {
            printf("Correct! You win!\n");
            break;
        } else if (guess < secretNumber) {
            printf("Too low! Try again.\n");
        } else {
            printf("Too high! Try again.\n");
        }
    }

    if (guess != secretNumber) {
        printf("You lose! The secret number was %d.\n", secretNumber);
    }

    return 0;
}
```

Analysis:

Gemini

Correctness:
The gemini code executes correctly, allowing 3 guesses before ending the program. It uses a fixed number as is required, and will tell if each guess is higher or lower. The 'break' stops the for loop when the correct number is guessed. The area where the code struggles is with user input errors, because there is no error handling. If the user inputs a letter, all three outputs are printed and the guess is marked as incorrect.

Execution Time:
Since this program uses a for loop, it can only run a finite number of times, this case being three. The number of operations is limited regardless of user input, so it would follow O(1) time complexity.

Space Complexity:
Space complexity is similar. It uses a fixed number of integer variables, including but not limited to the secret number and number of attempts. Regardless of input, it is always fixed, making space complexity O(1).

Maintainability:
This program is easy to understand because there are lots of comments. It uses accurate variable names and is organised in a way that makes it digestible. Because of this, the program is maintainable and readable. Some of the comments are rather long winded, but that just makes it look a little messy.

ChatGPT:

Correctness:

The ChatGPT code works as expected by giving the user up to three attempts to guess the secret number. It provides feedback after each guess by indicating whether the guess was too high or too low. When the correct number is entered, the `break` statement ends the while loop early. The program determines whether the user won by comparing the final guess with the secret number instead of using a separate winning flag. However, it does not include input validation, so entering a letter instead of a number could cause the program to behave incorrectly, similarly to the gemini.

Execution Time:

Since this program uses a while loop, it can only run a finite number of times, this case being three. The number of operations is limited regardless of user input, so it would follow O(1) time complexity. This is the same as the Gemini code because both programs have a maximum of three attempts.

Space Complexity:

Space complexity is similar to the Gemini code. It uses a fixed number of integer variables, including the secret number, guess, and number of attempts. Regardless of input, the amount of memory used stays fixed, making the space complexity O(1).

Maintainability:

This program is easy to understand because it is short and organized. It uses accurate variable names and does not have unnecessary code. Compared to the Gemini code, it has significantly fewer comments, which makes it look cleaner but may make it slightly harder for a beginner to understand. Overall, the program is readable and maintainable, although adding a few comments and making the secret number a constant could improve it.

Overall Comparison

One thing I didn't mention before is neither of the programs can handle overflow, so if I input the number 4294967303, which is 2^32 + secret number, they both mark as correct. Both programs have O(1) time and space complexity, and neither of them can handle input errors. The area where the codes differ is their looping style. GPT uses a 'while' loop where Gemini uses a 'for' loop. In this case, the for loop is more concise and it runs faster, but in the case that we wanted to give the user more guesses, we would have to change the for loop. Both methods work correctly, but in this case I prefer the for loop. The GPT version is more concise looking, but less readable because of the lack of comments. For a short program like this, that's okay, but it's generally a bad practice. This also makes me lean toward the Gemini code, but some of those comments are too cluttered. While revising the code, I'll aim for a happy medium that is readable but concise.

For these reasons, I selected the Gemini code to use and improve.

My Code

```
#include <stdio.h>

int main() {
    // Keep the secret number fixed as required by the assignment.
```

```c
    const int SECRET_NUMBER = 7;

    // Store the user's guess and track invalid input, then track the result.
    int guess = 0;
    int input_char;
    int won = 0;

    // Explain the valid range before requesting a guess.
    printf("Guess a number between 1 and 10.\n");

    // Allow up to three valid guesses.
    for (int attempt = 1; attempt <= 3; attempt++) {
        printf("Attempt %d/3. Enter your guess: ", attempt);

        // Check whether the input can be read as an integer.
        if (scanf("%d", &guess) != 1) {
            printf("Invalid input. Please enter a whole number.\n");

            // Remove the invalid characters before reading again.
            while ((input_char = getchar()) != '\n' && input_char != EOF) {
            }

            // Invalid input does not count as a guessing attempt.
            attempt--;
            continue;
        }

        // Stop immediately when the user guesses correctly.
        if (guess == SECRET_NUMBER) {
            printf("Correct! You win!\n");
            won = 1;
            break;
        } else if (guess < SECRET_NUMBER) {
            // Tell the user how to adjust a low guess.
            printf("Too low! Try again.\n");
        } else {
            // Tell the user how to adjust a high guess.
            printf("Too high! Try again.\n");
        }
    }

    // Report the secret number if none of the guesses was correct.
    if (!won) {
        printf("Sorry, you lost! The secret number was %d.\n", SECRET_NUMBER);
    }

    // Indicate that the program finished successfully.
    return 0;
}
```

My code checks for the validity of the input, and rejects it if it is not an integer. It also subtracts a guess to make only guesses that are integers count. The comments are concise and consistent, making the code readable but not messy looking. The main downfall of my program is it still uses scanf, so large numbers are not handled safely. To improve this, I could replace that with fgets and strtol, but I am not familiar with those commands at this time.